סמינר - Git and Virtual Machines

ליאור שילון

Seminar Outline

What We Will Cover Today:

- Git and Github
- Understanding Virtual Machines
- VMware Workstation Deep Dive and Installation
- Creating Your First Ubuntu VM
- Linux Fundamentals
- VM Best Practices and Tips

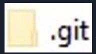
Part 1: Git and Github

Git

GIT היא מערכת ניהול גרסאות מבוזרת.
מעין מאגר העוקב אחר השינויים בקבצי הקוד בפרויקט שלנו.
מאפשר לראות הבדלים בקבצים בין גרסאות.

עובד בצורה לוקלית אך יכול להתחבר לשרתים מרוחקים.

Git - איך מתחילים?

Repository = מאגר
כאשר בחרנו פרויקט שאנו רוצים להגדיר לו את GIT כמנהל הגרסאות עלינו לפתוח Repository בתיקיית הפרויקט.
ע"י הפקודה: git init .
תיפתח תיקייה מוסת 

בתוך התיקייה של הפרויקט (אל תבצעו בה שינויים).
בתיקייה זו תישמר היסטוריית ה-commits שלכם, תגים, סימונים ועוד.

Git Branches

כשפותחים Repository נוצר ענף מרכזי: main.

פתיחת ענף:

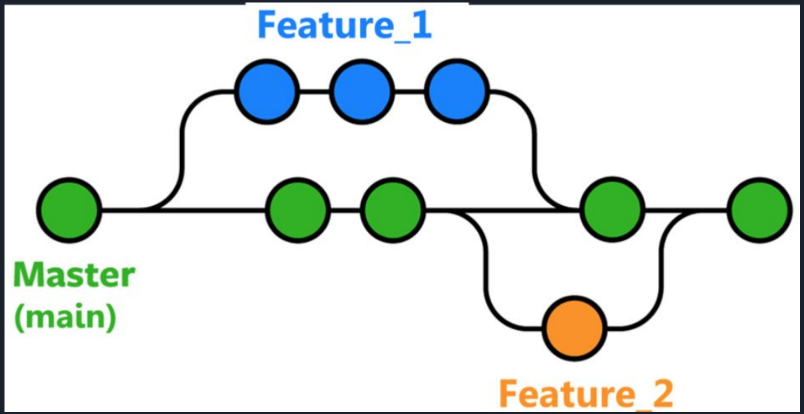
- git branch <branch_name>

מעבר בין ענפים:

- git checkout <branch_name>

מיזוג ענף לתוך הענף הנוכחי:

- git merge <branch_name>



התקנת Git

1. כנסו לקישור הבא: https://git-scm.com
2. לחצו על ההורדה

3. הורידו את הגרסה המתאימה למערכת ההפעלה שלכם: 32 סיביות או 64 סיביות (לרוב 64).

התקנת Git

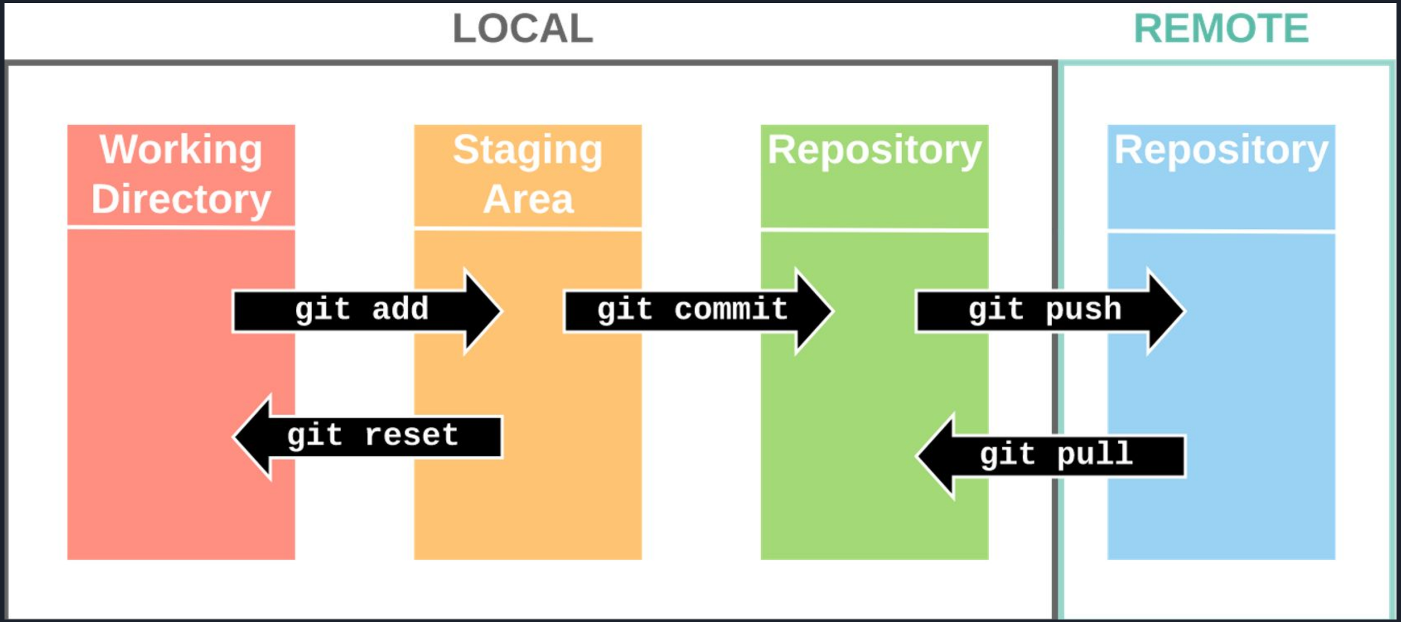
4. לאחר שההורדה הסתיימה, פתחו את התוכנה ולחצו על next עד שההתקנה מסתיימת.
5. פתחו חלון cmd חדש וכתבו בו את הפקודה git ולחצו enter.

```
C:\Users\shilo> git
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
           [--super-prefix=<path>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone             Clone a repository into a new directory
   init              Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add               Add file contents to the index
   mv                Move or rename a file, a directory, or a symlink
   restore           Restore working tree files
   rm                Remove files from the working tree and from the index
   sparse-checkout   Initialize and modify the sparse-checkout
```

GitHub

GitHub הוא שירות מבוסס ענן לניהול גרסאות ואחסון של פרויקטי פיתוח תוכנה המשתמשים במערכת Git. הוא מאפשר להעלות את הקוד לרשת ולעבוד עליו יחד עם אחרים, מה שמקל על שיתוף פעולה בצוותים ופרויקטים פתוחים. GitHub מספק כלים לניהול גרסאות, שילוב שינויים, ושיתוף פרויקטים בצורה מאורגנת ובטוחה.

איך הכל עובד


LOCAL
REMOTE
Working Directory
Staging Area
Repository
Repository
git add
git commit
git push
git reset
git pull

פקודות בסיסיות בGit

- הפקודה: git init
  מה היא עושה?
  מאתחלת מאגר Git חדש בספרייה.
  יוצר תיקייה נסתרת .git לניהול גרסאות.

- הפקודה: git clone <URL>
  מה היא עושה?
  מעתיקה מאגר קיים מהאינטרנט או ממחשב אחר.

- הפקודה: git status
  מה היא עושה?
  מציגה את מצב הקבצים במאגר.
  שימוש נפוץ:
  לראות אילו קבצים עודכנו, נוספו או נמחקו מאז השמירה האחרונה.

- הפקודה: git pull
  מה היא עושה?
  מורידה שינויים מהמאגר המרוחק לסביבה המקומית.

פקודות בסיסיות בGit

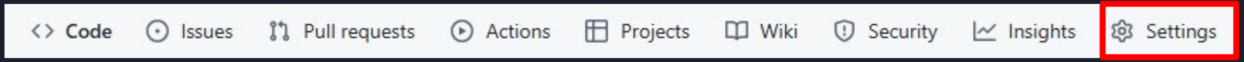
- הפקודה: git add <files> או git add .
  מה היא עושה?
  מוסיפה קבצים ל-staging area לפני השמירה.
  שימוש נפוץ:
  הכנת שינויים לשמירה באמצעות commit.

  הפקודה: git commit -m <message>
  מה היא עושה?
  שומרת את השינויים האחרונים כגרסה חדשה עם הודעה שמתארת את השינוי.
  חשיבות הודעת commit:
  עוזרת להבין את השינויים שנעשו לאורך ההיסטוריה של הפרויקט.

- הפקודה: git push
  מה היא עושה?
  מעלה את השינויים המקומיים למאגר מרוחק (GitHub).

הוספת collaborator

1. ב- github, לחצו על האייקון שלכם הנמצא בצד ימין למעלה.
2. בתפריט שנפתח לחצו על: your repositories.
3. בחרו ב-repository הרצוי.
4. לחצו על settings



5. בתפריט בצד שמאל של המסך, לחצו על collaborators.
6. לחצו על add people

7. חפשו לפי שם משתמש או אימייל .
8. משתמשים שנשלחה להם בקשת collaborator יקבלו מייל ובו יאשרו את הבקשה.

שימוש ב-github ו-git דרך Pycharm

Part 2: Virtual Machines

Understanding Virtual Machines

What is a Virtual Machine

A Virtual Machine (VM) is a software-based emulation of a physical computer system. It runs an operating system and applications just like a physical computer, but exists as a file (or set of files) on your host machine

The Virtualization Stack

Let's understand how virtualization works from the ground up

Layer	Component	Description
5	Applications	Software running inside the VM
4	Guest OS	Operating system installed in the VM
3	Virtual Hardware	Emulated CPU, RAM, disk, network
2	Hypervisor	VMware, VirtualBox, Hyper-V
1	Host OS	Your computer's operating system
0	Physical Hardware	CPU, RAM, Storage, Network

ליאור שילון

Understanding Virtual Machines

Types of Hypervisors

Type 1: Bare-Metal Hypervisor

- Runs directly on physical hardware
- No host OS required
- Examples: VMware ESXi, Microsoft Hyper-V Server, Citrix XenServer
- Used in: Data centers, enterprise environments

Type 2: Hosted Hypervisor

- Runs on top of a host operating system
- Easier to set up and use
- Examples: VMware Workstation, VirtualBox, VMware Fusion
- Used in: Development, testing, personal use

Understanding Virtual Machines

Why Use Virtual Machines

Real-World Applications

- Development & Testing:
  Test software on multiple OS versions
- Education:
  Learn new operating systems safely
- Security:
  Isolate potentially dangerous software
- Server Consolidation:
  Run multiple servers on one machine
- Disaster Recovery:
  Quick backup and restoration
- Legacy Software:
  Run old applications on modern hardware

Understanding Virtual Machines

Benefits of Virtualization

Benefit	Description	Example Use Case
Isolation	VMs are completely separated from host	Testing malware safely
Portability	VMs can be moved between hosts	Moving development environment
Snapshots	Save VM state at any point	Before installing updates
Resource Efficiency	Multiple VMs on one machine	Running web, database, app servers
Cost Savings	Reduce hardware requirements	One server instead of five

Virtual CPU (vCPU)

The hypervisor schedules virtual CPU time on physical CPU cores.
One physical core can support multiple vCPUs through time-slicing.

ליאור שילון

Understanding Virtual Machines

Virtual Memory (RAM)

- <u>Memory Allocation</u>: Each VM gets a portion of physical RAM
- <u>Memory Ballooning (dynamic memory management technique)</u>: Reclaim unused memory from VMs
- <u>Memory Overcommitment</u>: Allocate more virtual RAM than physical
- <u>Swapping</u>: Use disk when RAM is full (performance impact)

Virtual Storage

Storage Types:

- <u>Thick Provisioning</u>: Allocates all space (dedicated to the VM) immediately
- <u>Thin Provisioning</u>: Grows as needed (saves space)
- <u>Linked Clones</u>: Share base disk (of a VM) with parent (Another VM)

Understanding Virtual Machines

Virtual Networking

Network Mode	Description	Use Case
NAT	VM shares host's IP address	Need internet access, but no incoming connections
Bridged	VM gets its own IP on network	Full network participation (Hosting a web server or database)
Host-only	Private network with host	Isolated testing environment, where you don’t want external interference
Custom	Create virtual switches	Complex network topologies

VMware Workstation

VMware Workstation Overview

VMware Workstation is a Type 2 hypervisor that enables you to run multiple operating systems as virtual machines on a single Windows or Linux PC. It's the industry standard for development, testing, and deployment of applications.

Product	Type	Cost	Best For
VMware Workstation Pro	Type 2	Paid (~$200)	Professional developers
VMware Workstation Player	Type 2	Free for personal use	Students, home users
VMware Fusion	Type 2	Paid	Mac users
VMware ESXi	Type 1	Free/Paid editions	Enterprise servers

VMware Workstation

Key Features of VMware Workstation

Professional Features:

🔗 VM Cloning: Full and linked clones
📸 Snapshots: Save and restore VM states
🌐 Advanced Networking: Virtual network editor
🔒 Encryption: Secure your VMs
🔄 VM Sharing: Share VMs with Workstation Server to allow remote connections
🛠️ Developer Tools: Integration with Visual Studio, Eclipse (Build, test, and debug applications directly inside VMs from your IDE)

System Requirements

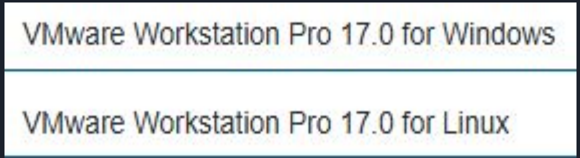
⚠️ **Minimum Requirements for VMware Workstation 17**:

- CPU: 64-bit x86 Intel or AMD processor with 1.3GHz or faster
- RAM: 2GB minimum (4GB recommended, 8GB+ for multiple VMs)
- Disk: 1.2GB for application + space for VMs
- OS: Windows 10/11 or Ubuntu 20.04+
- Important: CPU must support virtualization (Intel VT-x or AMD-V)

Downloading VMware Workstation

Download

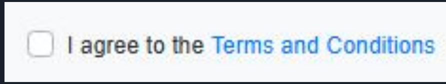
Step 1: Create a Broadcom account using your email - > link
Step 2: After creating your account, make sure to sign in -> link
Step 3: Go to the VMware Workstation 17 Pro download page -> link
Step 4: Select the version matching you OS



Step 5: Select the last release

Release	Release Level Info
17.6.4	533272
17.6.3	528496

Step 6: You won’t be able to download until you view the “Terms of Conditions” and check the “I agree” box

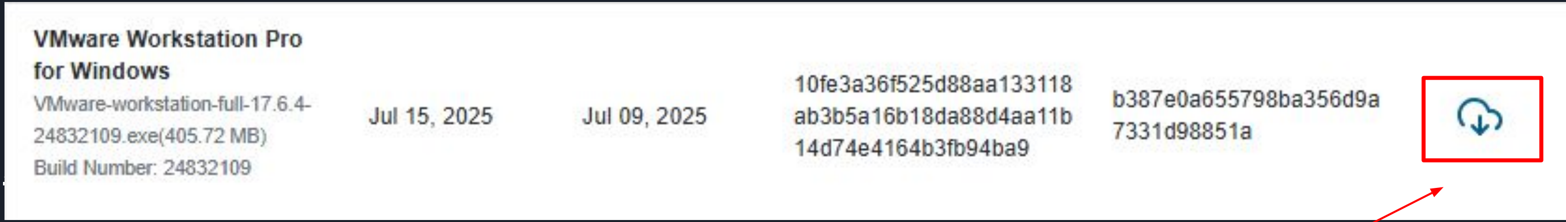


ליאור שילון

Downloading VMware Workstation

Download

<u>Step 7:</u> Download by pressing marked button.

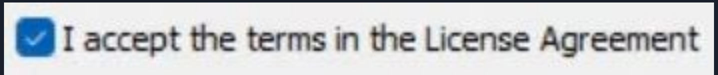


Step 8: Complete the verification process and press the download button again.
The download will start.

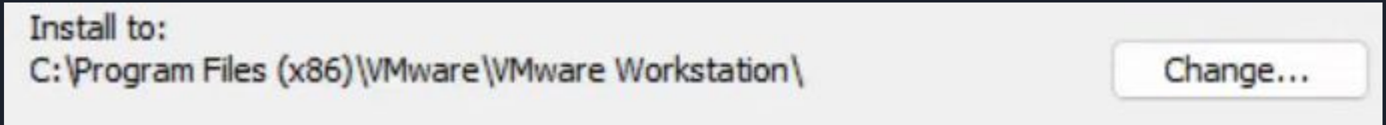
Installing VMware Workstation

Installation

Step 1: After downloading the execution file, tun it.
Step 2: Press next and accept the License Agreement



Step 3: You can change the installation path or use the default one (useful if you have space problems)

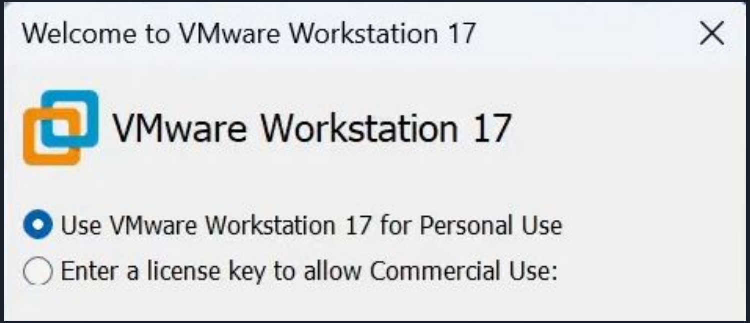


Step 4: Next, next and click install
Step 5: After installation is complete, open the software

Installing VMware Workstation

Installation

<u>Step 6:</u> Check the "Personal Use" radio button

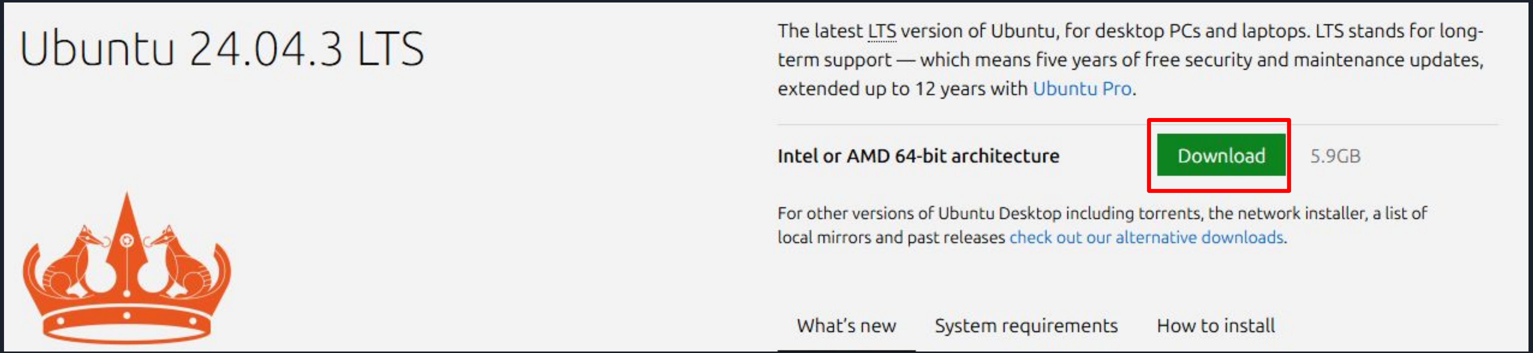


You might need to enable virtualization on your computer through the BIOS

Creating Your First Ubuntu VM

Downloading Ubuntu ISO

Step 1: Go To Ubuntu website and press the download button (You will need to provide the path to this file when setting up the virtual machine)



Creating the VM

Step 2: A 10 minutes YouTube video showing how to create the machine with the ubuntu image on it. -> link

ליאור שילון

Part 3: Linux Fundamentals

ליאור שילון

Introduction to the Linux Terminal

What is the Terminal

The terminal (or shell) is a text-based interface to interact with the Linux operating system. It's powerful, efficient, and essential for system administration and development

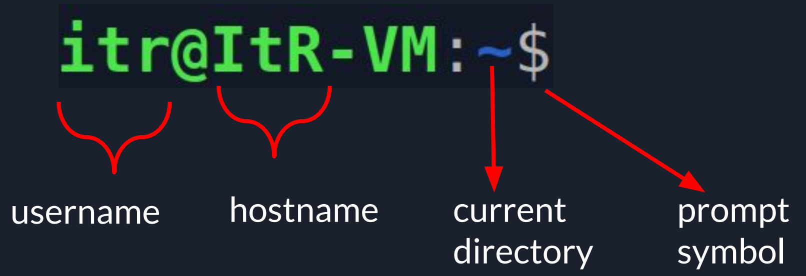
Opening the Terminal

Multiple ways to open terminal in Ubuntu:

1. Keyboard Shortcut: Ctrl + Alt + T
2. Application Menu → Type "Terminal
3. Right-click on Desktop → "Open in Terminal"

Introduction to the Linux Terminal

Understanding the Shell Prompt



<u>Prompt symbols:</u>
$ = Regular user
# = Root user (administrator)

The tilde (~) represents your home directory
Full path: /home/itr (the username)

ליאור שילון

Introduction to the Linux Terminal

Basic Navigation Commands

```
# pwd - Print Working Directory
student@ubuntu-vm:~$ pwd
/home/student

# ls - List directory contents
student@ubuntu-vm:~$ ls
Desktop  Documents  Downloads  Music
Pictures  Videos

# ls with options
ls -l  # Long format with details
ls -a  # Show hidden files (starting with .)
ls -la  # Combine options
ls -lh  # Human-readable file sizes
```

```
# cd - Change Directory
cd /path/to/directory  # Absolute path
cd Documents     # Relative path
cd ..       # Parent directory
cd ../..      # Two levels up
cd ~       # Home directory
cd -       # Previous directory

# Examples:
cd /etc      # Go to /etc directory
cd ~/Documents     # Go to Documents in home
cd       # Go to home (same as cd ~)
```

File and Directory Operations

Creating Files and Directories

```
# mkdir - Make Directory
mkdir test_folder
mkdir -p path/to/nested/folders   # Create parent directories

# touch - Create empty file or update timestamp
touch newfile.txt
touch file1.txt file2.txt file3.txt  # Multiple files

# Creating files with content
echo "Hello World" > hello.txt     # Create with content
echo "New line" >> hello.txt     # Append to file
cat > myfile.txt          # Print the file content to shell
```

File and Directory Operations

Creating Files and Directories

```
# nano - Text editor
nano myfile.txt            # Edit/create file
# Ctrl+O -> Enter to save, Ctrl+X to exit

# Advanced directory creation
mkdir -p project/{src,docs,tests}/{subfolder1,subfolder2}
# Creates complex directory structure
```

File and Directory Operations

Copying, Moving, and Deleting

```
# cp - Copy files/directories
cp source.txt dest.txt          # Copy file
cp -r source_dir dest_dir       # Copy directory recursively
cp -i file.txt backup.txt       # Interactive (confirm overwrite)
cp -p original.txt copy.txt     # Preserve attributes

# mv - Move/Rename files
mv oldname.txt newname.txt      # Rename file
mv file.txt /path/to/directory/ # Move file
mv *.txt Documents/             # Move all .txt files

# rm - Remove files/directories
rm file.txt                     # Delete file
rm -r directory/                # Delete directory recursively
rm -f file.txt                  # Force delete (no prompt)
rm -rf directory/               # Force recursive (DANGEROUS!)
rm -i *.txt                     # Interactive deletion
```

⚠️ Safety Tips:

- CAREFUL WITH `rm -rf` → Deleted files are gone forever
- Always use `ls` first to verify what you're about to delete
- Use `rm -i` for interactive confirmation
- Never run `rm -rf /` (destroys entire system)

Essential Linux Commands

Viewing and Searching Files

```
# cat - Concatenate and display files
cat file.txt            # Display entire file
cat file1.txt file2.txt # Display multiple files
cat -n file.txt         # Show line numbers

# less - Search  through files
less largefile.txt      # View with scrolling
# Navigation in less:
# Space = next page, b = previous page
# / = search forward, ? = search backward
# q = quit

# head/tail - View beginning/end of files
head file.txt           # First 10 lines
head -n 20 file.txt     # First 20 lines
tail file.txt           # Last 10 lines
```

Essential Linux Commands

Viewing and Searching Files

```
# grep - Search text patterns
grep "pattern" file.txt          # Search for pattern
grep -i "pattern" file.txt       # Case-insensitive
grep -r "pattern" directory/     # Recursive search
grep -n "pattern" file.txt       # Show line numbers

# find - Search for files
find . -name "*.txt"             # Find .txt files
find / -size +100M 2>/dev/null   # All files larger than 100MB on the system
find . -mtime -7                 # Modified in last 7 days
find . -type d -name "test*"     # Find directories

# wc - Word count
wc file.txt                      # Lines, words, characters
wc -l file.txt                   # Line count only
wc -w file.txt                   # Word count only
```

Essential Linux Commands

Process Management

```
# ps - Process status
ps              # Current shell processes
ps aux          # All processes, detailed
ps aux | grep firefox # Find specific process

# top/htop - Interactive process viewer
top             # Real-time process monitor
# Press q to quit, k to kill process

htop            # Better interface (install: sudo apt install htop)

# kill - Terminate processes
kill PID        # Send TERM signal
kill -9 PID     # Force kill (SIGKILL)
killall firefox # Kill by name
```

Package Management with APT

APT (Advanced Package Tool)

Ubuntu's package management system for installing, updating, and removing software from repositories.

Basic APT Commands

```
# Update package list (always do this first!)
sudo apt update

# Upgrade installed packages
sudo apt upgrade        # Upgrade packages

# Installing software
sudo apt install package_name
sudo apt install git vim curl   # Multiple packages
sudo apt install -y package   # Auto-yes to prompts

# Removing software
sudo apt remove package_name   # Remove package
```

Part 4: VM Best Practices and Tips

Performance Optimization

Performance Optimization

VM Performance Tips:

- 📊 Allocate appropriate resources (not too much, not too little)
- 💾 Use SSD for VM storage when possible
- 🔧 Install VMware Tools/Open VM Tools
- 🎮 Enable hardware acceleration for graphics
- 📦 Use thin provisioning for disk space
- 🔄 Disable unnecessary startup services
- 🧹 Clean package cache regularly: sudo apt clean

Snapshot Strategy

Snapshot Strategy

When to take snapshots:

- Before major updates
- Before configuration changes
- After successful setup milestones
- Before experimenting with system

Snapshot best practices:

- Name descriptively: "Pre-Apache-Install-2024-01-15"
- Add descriptions
- Delete old snapshots (they consume space)

VMware Snapshot Commands (from host):

- `vmrun -T ws snapshot "path/to/vm.vmx" "Snapshot Name"`
  → Take a snapshot of the VM's current state
- `vmrun -T ws listSnapshots "path/to/vm.vmx"`
  → List all snapshots for this VM
- `vmrun -T ws revertToSnapshot "path/to/vm.vmx" "Snapshot Name"`
  → Revert the VM back to a specific snapshot